Python Portfolio — Data Flow Analysis

Project: Python Portfolio
Technology: Python, FastAPI, SQLAlchemy, SQLite, Pydantic, JWT
Location: `/home/dominic/Documents/python-portfolio`
Report #: 15/20

Request Flow

```
Client → Auth Router → JWT
      → Shortener Router → SQLite
      → Blog Router (GraphQL) → SQLite
      → Chat Router → OpenAI
      → Queue Router → Task Queue
      → RAG Router → ChromaDB + OpenAI
```

Data Flow Analysis

Request Data Flow

```
Client Request
    │
    ▼
CORS Middleware ——→ Check Origin Headers
    │
    ▼
Auth Middleware ——→ Decode JWT → Extract User
    │
    ▼
Router ——→ Match Path → Call Handler
    │
    ▼
Handler
    ├——→ Validate Input (Pydantic)
    ├——→ Query/Write DB (SQLAlchemy)
    ├——→ Call External API (httpx)
    └——→ Format Response
    │
    ▼
JSON Response ←—— Serialize (Pydantic)
```

Database Transaction Flow

```
Begin Transaction
    │
    ▼
Query ──→ SELECT with filters
    │
    ▼
Validate ──→ Check business rules
    │
    ▼
Mutate ──→ INSERT/UPDATE/DELETE
    │
    ▼
Commit ──→ On success
Rollback ──→ On error
```

WebSocket Data Flow (Queue Module)

```
Client opens WebSocket
    │
    ▼
Server accepts connection
    │
    ▼
Task created → Server pushes progress updates
    │
    ▼
Client receives: {"progress": 45, "status": "running"}
    │
    ▼
Task completes → {"progress": 100, "status": "completed"}
    │
    ▼
WebSocket closes
```

External API Data Flow (AI Chat Proxy)

```
Client → POST /v1/chat/completions
    │
    ▼
Proxy → Add API key → Forward to api.openai.com
    │
    ▼
OpenAI → SSE stream (tokens)
    │
    ▼
Proxy → Forward stream as-is
    │
```

```
    ▼
Client → Parse SSE → Render tokens incrementally
```

Data Transformations

1. **URL shortening:** Long URL → short_code (base62 hash)
2. **Auth:** Password → bcrypt hash → DB
3. **RAG:** PDF → Text → Chunks → TF-IDF vectors
4. **Queue:** Task request → DB row → WebSocket progress
5. **Chat:** Message → JSON → API call → SSE stream